AI Compiler Performance Engineering

Auto-Loops Books — ch13

Contents

1 Core Theory: AI Compiler Optimization Principles **1**

1.1 tiling theory 2

1.2 fusion theory 6

1.3 layout theory 9

1.4 parallel scheduling 12

1.5 GPU CPU NPU split 16

1.6 Key Takeaways 20

1.7 Conclusion 21

Bibliography **23**

Chapter 1

Core Theory: AI Compiler Optimization Principles

What makes an AI compiler "correct" for production decode—pass ordering that looks elegant on a whiteboard, or schedules that measurably cut bytes/token and launches/token on the SKU you ship? Part VI opens with *Core Theory*: the optimization principles that sit *below* Chapter 12 execution modes and *above* dialect-specific backends (MLIR, Glow, TVM) (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Every section binds IR transforms to residency on GPU, CPU, and NPU—not abstract big-O (Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor reports 1.259× median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that tiling and fusion must extend into bytes, not replace (Chen, 2026; NVIDIA, 2024). Compiler theory is how those counters move together: tile so data stays on-chip, fuse so boundaries disappear, layout so hardware reads coalesce, parallelize so batch-1 decode still saturates memory paths, and split backends so one IR lowers per target without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

This chapter is deliberately *theory-first*: no MLIR syntax yet (Chapter 14), no manual CUDA (Part VII). The goal is a shared vocabulary for pass designers—when someone proposes a new fusion rule or tile heuristic, Table 1.1 names which counter must move and which prior chapter constraint applies (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Industrial teams that skip

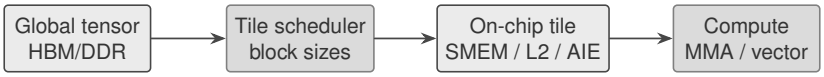


Figure 1.1: Tiling theory: map global decode tensors into on-chip tiles before compute.

this layer debate dialect patches for weeks while bytes/token flatlines because layout and tiling were never co-designed (Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Fregly-style goodput analysis applies throughout: every transform is judged on batch-1 decode at representative context buckets, not on square GEMM microbenchmarks or prefill-only traces (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026). The five sections mirror how production compilers order concerns—tile shapes constrain fusion legality; layout determines whether fused regions can load coalesced data; parallel axes decide whether extra blocks help bandwidth; backend split decides which target owns each residency class (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Readers coming from classic optimizing-compiler courses should expect different objectives: decode LLM compilers optimize *moved bytes* and *launches* under numerics constraints, not loop nest locality alone (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When in doubt, re-run the Chapter 1 counter worksheet on the same model checkpoint before and after a pass change—theory without measurement is slideware (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

1.1 tiling theory

Tiling theory asks which sub-blocks of weights, activations, and KV cache fit in registers, shared memory, L2, or NPU tile SRAM for each decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022). FlashAttention is the canonical IO-aware tiling result for attention: recomputation and block-wise softmax avoid materializing P in HBM (Dao *et al.*, 2022, 2023). The compiler question is general—every matmul, norm, and MLP in decode needs a tile shape that respects bandwidth and capacity, not only MMA convenience (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026).

On Hopper, tiling couples to TMA: async copies move KV tiles while MMA consumes prior tiles (NVIDIA, 2022; SemiAnalysis, 2024). Wrong tile sizes leave

TMA underutilized or spill partial outputs to HBM, showing up as bytes/token regression versus a smaller but legal tile (Chen, 2026; Chen and YiRage Contributors, 2026). On XDNA/AIE, tiling is static: DMA chains and tile slots are fixed at compile time; "dynamic" GPU tiles do not lower verbatim (AMD, 2024b,a; Chen, 2020). CPU decode tiling is cache blocking—NUMA-local KV and blocked GEMMs keep lines hot across norm–matmul chains (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Analytic tile sizing. A first-principles decode tile budget starts from hidden size d, head count h, KV dtype bytes b_{kv}, and context length L: bytes moved per token scale with reading prior KV plus writing new KV slots (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025). Tile width along L trades SMEM for trip count; tile width along d trades MMA efficiency for reload width (Dao *et al.*, 2023; SemiAnalysis, 2024; Chen, 2026). Compilers should expose these trade-offs in IR annotations so autotuners search a *legal* polytope rather than all factorizations of L (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). When analytic estimates disagree with profiles by more than a few percent, trust profiles—roofline models omit allocator, paging, and framework overhead (Chen, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Compiler passes should emit tile metadata in IR (tile sizes, double-buffer depth, residency class) so backends reject illegal schedules before codegen (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Autotuners that search FLOPs-only tile shapes without bytes/token feedback recreate prefill wins and decode losses (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Profile each candidate tile on BS=1 decode cells, not only square GEMM microbenchmarks (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024).

Prefill vs decode tiling. Prefill benefits from large token-parallel tiles that amortize weight loads across many queries; decode at BS=1 reuses weights once per step and instead fights KV reload along the sequence axis (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). A tile policy tuned on prefill shapes often selects block sizes that maximize MMA occupancy while increasing KV trips—exactly the regression Memory-Floor documents when bytes/token stay high despite faster attention kernels (Chen, 2026; Vellaisamy *et al.*, 2026). Compiler tile search must therefore tag phases explicitly: prefill buckets, decode buckets, and speculative-draft buckets when applicable (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Double buffering and pipeline depth. Hopper pipelines pair TMA loads with MMA compute; tile theory includes how many stages fit in shared memory without spilling partial softmax or norm statistics (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022). Shallow pipelines underutilize memory bandwidth; deep

pipelines overflow SMEM and force silent HBM spill (Chen, 2026; Chen and YiRage Contributors, 2026). On AIE, pipeline depth is fixed at compile time—there is no runtime knob to "add another buffer" when decode context grows (AMD, 2024b,a; Chen, 2020).

Autotuning discipline. TVM-style autotuners historically optimized for peak FLOPs on convolution and dense GEMM shapes (Goodfellow *et al.*, 2016; Chen, 2020). LLM decode autotuning must swap the objective: minimize bytes/token subject to numerics and legality constraints from Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Every autotune candidate should log tile dimensions *and* measured HBM traffic per token—otherwise teams ship tiles that win on roofline FLOPs charts and lose on fleet p99 (Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024).

Multi-hardware delta. Identical FLOP count with different tile policies can differ by an order of magnitude in DDR traffic on XDNA versus TMA-backed Hopper tiles (AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 1.1.1. Decode cell (BS=1, ctx=2048): shrinking attention block size to fit shared memory may *increase* loop iterations and bytes/token if KV reloads dominate—tile theory must co-optimize residency and trip count (Dao *et al.*, 2022; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Industrial tile review checklists include: SMEM/TMA budget worksheet, KV bytes per trip, prefill vs decode bucket tags, and autotune objective function (SemiAnalysis, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Teams that approve tile changes without decode profiles often ship kernels that win on nsys "compute" rows and lose on "dram" rows (Vellaisamy *et al.*, 2026; Chen, 2026; Dao *et al.*, 2022; AMD, 2024b; Goodfellow *et al.*, 2016).

Residency classes before tile shapes. The first tiling decision is not a number but a class: every tensor in the decode graph must be classified as register-resident, shared-memory or tile-SRAM resident, L2-resident, or global-memory streamed (SemiAnalysis, 2024; NVIDIA, 2022; Chen, 2026). Only after classification does the pass choose concrete tile widths that satisfy the class boundaries; choosing MMA-preferred widths first and hoping residency follows is how fused graphs silently spill (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). The classification itself is hardware-derived—Hopper exposes shared memory and L2 as separate

budgets, XDNA fixes tile addresses at compile time, and CPU treats cache lines as the residency unit (AMD, 2024b,a; Goodfellow *et al.*, 2016). IR should carry the class as an attribute so a later pass cannot demote a register tensor to shared memory without a legality review (Chen, 2020; Davies *et al.*, 2025).

Tile sweep methodology. Tile autotuning becomes reproducible when the search is bounded by the residency classification first and measured on decode counters second (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). Generate candidates by varying one axis at a time—KV-block width, head-block width, double-buffer depth—while holding the other axes at the analytic default; log bytes/token and ms/token for every candidate at the same context bucket (Dao *et al.*, 2023; SemiAnalysis, 2024). Narrow the sweep around the best bytes/token point, not the best SM-utilization point, because batch-1 decode rewards fewer and wider memory transactions (Chen, 2026; Vellaisamy *et al.*, 2026). Keep the full sweep table in CI artifacts so a later chip-model update can re-run the same procedure without re-deriving the methodology (Chen and YiRage Contributors, 2026; Chen, 2020).

Decode-specific tile invariants. Three invariants separate decode tiling from prefill tiling. First, weight tiles are read once per token, so they should stream through the largest reusable footprint rather than be re-tiled around a batch of queries (Kwon *et al.*, 2023; Davies *et al.*, 2025). Second, KV tiles are read on every new token, so their width along the sequence axis directly sets bytes/token and the shared-memory high-water (Dao *et al.*, 2022, 2023). Third, softmax and norm statistics must remain resident across tile boundaries; a tile that invalidates their lifetime is illegal regardless of how fast it loads (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). These invariants belong in the compiler test suite as negative tests: a pass that proposes an illegal decode tile should fail CI with the invariant name (Chen, 2026, 2020).

Capacity ledger as review artifact. Every tiling PR should carry a capacity ledger: tensor names, assigned residency classes, byte estimates at the shipped context bucket, and the measured bytes/token from the decode profile (SemiAnalysis, 2024; Chen, 2026; Chen and YiRage Contributors, 2026). The ledger turns an abstract tile change into a testable claim—if the ledger says an activation is shared-memory resident but the profile shows DRAM traffic growing with that tensor's size, the claim is falsified in review (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Compiler-generated plans should emit the ledger automatically from IR

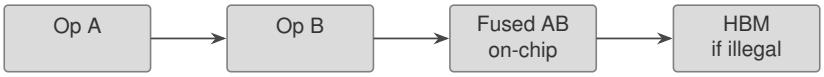


Figure 1.2: Fusion theory: merge producer–consumer ops when residency allows; else spill.

metadata so it cannot go stale (Chen, 2020; Chen and YiRage Contributors, 2026). Reviewers should reject tile PRs whose ledger and profile disagree, because that disagreement is the earliest symptom of silent spill (Chen, 2026; Vellaisamy *et al.*, 2026).

1.2 fusion theory

Fusion theory governs when operator boundaries may disappear into MegaKernels or fused subgraphs (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each unfused boundary exports activations to HBM; Chapter 1 counted the result as launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023). Fusion is not "always on"—it is constrained by register/shared-memory budgets, softmax carrier lifetimes (Chapter 10), and graph capture legality (Chapter 12) (Dao *et al.*, 2022; NVIDIA, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Greedy fusion (merge any adjacent ops) often illegalizes attention–MLP seams: online softmax state (m, ℓ, o) must survive inside the fused region or bytes explode silently (Dao *et al.*, 2022, 2023; Chen, 2026). Compiler cost models should penalize HBM round trips and launches, not only instruction count (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). YiRage-style legality checks encode Chapter 2 iron rules as IR predicates so fusion plans fail in compile time rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a).

Fusion boundaries as contracts. Treat each candidate fusion edge as a contract between producer and consumer: the producer promises not to materialize to HBM; the consumer promises register/SMEM budget to consume in-place (Chen and YiRage Contributors, 2026; Chen, 2026; Dao *et al.*, 2022). When either side breaks the contract—for example LayerNorm statistics exported because the following GEMM needs a different layout—fusion must split (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture (Chapter 12) records whatever fusion boundaries exist; it cannot invent new ones (NVIDIA, 2024; Chen, 2026).

Horizontal vs vertical fusion. *Horizontal* fusion merges operators at the same tensor rank (norm + linear + activation). *Vertical* fusion merges stages

along the decode pipeline (attention block + MLP block into one MegaKernel region) (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Vertical fusion delivers the largest bytes/token wins but hits legality first; horizontal fusion is easier but may leave launches/token high (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025). Compiler heuristics should attempt vertical plans only after horizontal plans pass carrier and layout checks (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024).

Fusion and execution modes. Chapter 12 maps eager, graph, and MegaKernel modes to launch and byte counters; fusion theory explains *which* boundaries each mode can erase (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture records existing fusion; MegaKernel mode requires fusion plans that satisfy carrier legality (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024). Teams stuck in graph mode with high bytes/token are blocked by fusion theory, not by CUDA Graph APIs (Chen, 2026; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Document fusion-closed regions in runbooks so on-call engineers know whether to tune capture buckets or rewrite fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Cost model sketch. A minimal fusion cost model scores each plan as $\alpha \cdot \text{bytes} + \beta \cdot \text{launches} + \gamma \cdot \text{spill_penalty}$, with spill penalty derived from residency metadata rather than FLOPs (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Tuning (α, β, γ) per SKU is acceptable; tuning without bytes/token telemetry is not (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hopper may fuse QKV+softmax+proj when shared memory permits; XDNA may require separate static subgraphs linked by DMA; CPU fusion is often cache-scope fusion across norm–linear chains (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023).

Example 1.2.1. Fusing LayerNorm with the following GEMM removes one full-width activation write/read per layer per token—often larger decode win than tuning MMA tile width alone (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

MoE layers add gather/scatter fusion constraints: expert routing may remain host-visible while MLP experts fuse internally—fusion theory must specify *which* subgraph is fusion-closed (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a; Davies *et al.*, 2025; Chen, 2026).

Fusion regression tests should diff IR fusion boundaries against a golden eager graph: any new HBM edge between previously fused ops is a release blocker unless documented as intentional (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). This is how compiler teams prevent "silent unfusion" when dialect refactors reorder passes (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024a,b).

Fusion plans as IR contracts. A fusion plan should be representable as a first-class IR object: the fused region, the boundaries it removes, the buffers it keeps on-chip, and the numerical assumptions it makes (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Without that object, a pass that merges two ops and a pass that merely reorders them look identical in review, and silent unfusion can hide for releases (Chen, 2026; Vellaisamy *et al.*, 2026). The contract should be diffable in code review: a PR that changes fusion boundaries changes the contract diff, and the byte sheet must move with it (Davies *et al.*, 2025; Chen, 2020). Compilers that encode contracts this way also enable rollback at the region level instead of at the whole-kernel level (Chen and YiRage Contributors, 2026; Chen, 2026).

Layout-changing producers. Many fusion opportunities die at layout conversions rather than at compute: a producer that emits a transposed or interleaved activation forces the consumer to either reshape in place or load strided (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Fusion theory must therefore include a layout negotiation step where producer and consumer agree on one layout before the boundary disappears (Chen, 2020; Chen and YiRage Contributors, 2026). When no common layout exists within the on-chip budget, splitting the fusion is cheaper than inserting an HBM transpose, but the split must be explicit in the contract so its bytes are counted (Chen, 2026; Vellaisamy *et al.*, 2026). Pass ordering that runs layout legalization before fusion search prevents these negotiations from happening as emergency rewrites at codegen (Chen, 2020; AMD, 2024b).

Fusion discovery order. Production pass pipelines should discover fusion in a fixed order: elementwise producers into their dominant consumers first, norm and scale operators into the GEMMs that follow, attention seams only after online-carrier legality is proven, and MoE routing last because it changes control flow (Dao *et al.*, 2022; Liu *et al.*, 2024; Chen, 2020). Each stage should emit a legality report that names the buffers preserved and the counters expected to move (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026). This ordering prevents a greedy pass from consuming a seam that later, more valuable fusion needs for carrier residency (Dao *et al.*, 2022; Davies *et al.*, 2025). It also gives engineers a vocabulary for regressions: a fusion that disappears between releases can be traced to the stage that owns that seam (Chen and YiRage Contributors, 2026; Chen, 2020).

Spill accounting discipline. Spills are not an error if they are counted; they are a bug when they are invisible (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Fusion passes should emit a spill event every time a planned on-chip buffer falls back to global memory, with the tensor name and byte width attached (Chen and YiRage Contributors, 2026; Chen, 2020). Production dashboards should alert when spill bytes grow across compiler versions, because a silent unfusion and an over-eager new fusion produce the same spill signature (Chen, 2026; Vellaisamy *et al.*, 2026). Reviewers should ask fusion PRs to show the before-and-after spill ledger; a PR that reports only kernel count reductions has not yet demonstrated a bytes win (Davies *et al.*, 2025; SemiAnalysis, 2024).

1.3 layout theory

Layout theory decides how tensors are stored and traversed: contiguous vs blocked layouts, KV cache page tables, head/interleaved formats, and alignment for TMA/vector loads (Kwon *et al.*, 2023; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022). A perfect fusion plan fails if layout forces strided HBM access—bytes/token rise while kernels look "fused" in the IR (Chen, 2026; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020).

PagedAttention (vLLM) is layout engineering for KV capacity: non-contiguous physical pages with logical continuity reduce fragmentation without changing model math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Compilers must treat page table indirection as a first-class layout transform, not an runtime hack outside IR (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). On NPU backends, layout often *is* the schedule—static DMA descriptors

replace pointer chasing (AMD, 2024b,a; Goodfellow *et al.*, 2016).

Phase-specific layouts. Some teams store KV in a layout optimized for prefill writes (wide vector stores along sequence) and transposes for decode reads (head-local access) (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Transpose costs bytes; the compiler must charge them in the cost model and prefer layouts that avoid per-token transposes when ctx grows (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024; Dao *et al.*, 2023). When a layout pass cannot prove decode legality, split layouts per phase rather than forcing one format (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Layout passes interact with tiling: blocked weights pair with blocked GEMM tiles; NHWC vs NCHW choices change vectorization on CPU and GPU differently (Goodfellow *et al.*, 2016; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; AMD, 2024a). Decode compilers should log layout transforms in pass metadata so regressions trace to reorder vs fusion vs tile changes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

KV layout and paging. Decode throughput scales with how cheaply each new token reads historical KV (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Contiguous KV is simple but fragments GPU memory at long context; paged layouts add indirection cost that must be amortized by fewer reallocations and higher batch utilization (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compiler layout passes should lower page-table updates into explicit IR ops so backends can fuse gather/scatter with attention tiles where legal (Chen and YiRage Contributors, 2026; Chen, 2020).

Weight layout and quantization. INT4/FP8 weight layouts (block scales, interleaved scales) are layout transforms before they are "quantization features" (Liu *et al.*, 2024; Davies *et al.*, 2025; SemiAnalysis, 2024). A fusion plan that assumes FP16 contiguous weights will fail silently when weights are block-scaled—loads become gather-bound and bytes/token rise (Chen, 2026; Chen and YiRage Contributors, 2026). Layout legalization must run *before* fusion search, not as a post-pass patch (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Alignment and vector width. TMA and vector units prefer aligned 128-byte lines; misaligned KV or activation tensors force uncoalesced access patterns that no tile size fixes (SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016). CPU decode benefits from cache-line alignment across head dimensions; NPU backends often require fixed stride multiples baked into DMA descriptors (AMD, 2024b,a; Chen, 2020). Layout theory therefore includes alignment as a first-class constraint in IR, not a codegen afterthought (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Multi-hardware delta. GPU layouts optimize for coalesced 128B lines; CPU layouts optimize for cache line reuse across batch-1 thin GEMMs; NPU layouts fix byte addresses at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024a).

Example 1.3.1. Switching KV layout to improve prefill vectorization can regress decode if batch-1 attention loads become gather-bound—layout theory requires phase-specific profiling (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024; AMD, 2024a).

Layout CI should snapshot tensor strides and alignment for KV, QKV weights, and MLP weights at each context bucket (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024b,a). Stride drift without corresponding fusion changes is a common source of "mysterious" bytes/token regressions across compiler versions (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Layout canonicalization and legality. Every phase should have a canonical layout derived from its access pattern, with explicit ops for any conversion rather than hidden runtime transposes (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). Legality in layout terms means: the chosen strides are aligned for the target's vector or DMA width, the tensor fits its assigned residency class, and no consumer requires an implicit reshape (NVIDIA, 2022; AMD, 2024a; Davies *et al.*, 2025). A layout pass that cannot prove these three properties should refuse to emit rather than insert a conversion that doubles bytes/token (Chen,

2026; Vellaisamy *et al.*, 2026). CI should snapshot the canonical layout of KV, QKV weights, and MLP weights per context bucket so drift appears as a first-class regression (Chen, 2020; Goodfellow *et al.*, 2016).

Paged KV layout decisions. Paging changes the layout question from "where is the next KV row" to "which page does the block walk touch," and the answer determines whether attention loads coalesce (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Page size should be chosen against the head dimension and the on-chip staging width: pages that are too small scatter the block walk, while pages that are too large waste capacity on partial occupancy (SemiAnalysis, 2024; Chen, 2026). The compiler should expose the page-table walk as an explicit IR pattern so fusion can keep gather and attention in one region where the target allows it (Chen and YiRage Contributors, 2026; Chen, 2020). On XDNA, page indirection may need to become a static descriptor list because runtime pointer chasing is not a legal DMA pattern (AMD, 2024b,a).

Weight layout conversion timing. Quantized and block-scaled weights should be converted once at load or export time, never per decode step, and the conversion cost must appear in the same ledger as activation traffic (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). A compiler that treats weight conversion as a free pre-pass hides a real cost when models are swapped at runtime or when context buckets share one weight copy (Chen, 2020; Chen and YiRage Contributors, 2026). Layout legalization should verify that every quantized tensor has a single canonical runtime layout and that no fusion plan inserts a second conversion (Davies *et al.*, 2025; Chen, 2020). Runtime weight swaps are where layout bugs surface as mysteriously slow first tokens; the fix belongs in the pass pipeline, not in cache tuning (Vellaisamy *et al.*, 2026; Chen, 2026).

1.4 parallel scheduling

Parallel scheduling maps work to SMs, cores, or AIE arrays—including Flash-Decoding's parallelization over KV sequence length at batch-1 (Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Prefill parallelizes over tokens; decode parallelizes over heads, KV blocks, or expert routes while issuing one new token per step (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

The failure mode is "parallelism for idle hardware": launching extra blocks that contend for HBM bandwidth without increasing useful bytes delivered per ms/token (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Schedulers should be bandwidth-aware—when R_{floor} is high, additional parallel splits hurt (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025). When launches/token dominate (TaxBreak regime), parallel capture inside CUDA Graphs or fused kernels may help even at moderate bandwidth (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Prefill/decode scheduler split. Prefill schedulers maximize token-parallel work; decode schedulers maximize memory efficiency per new token (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Sharing one scheduler heuristic across phases is a common source of decode regressions after prefill optimizations (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Compiler IR should expose parallel axes (batch, head, KV block, expert) so backends legalize splits per target (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Static NPU schedules often fix parallelism at compile time; GPU schedules retain dynamic grid sizing within capture buckets (AMD, 2024b; NVIDIA, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Occupancy vs bandwidth. GPU textbooks emphasize occupancy; decode compilers must ask whether extra warps increase *useful* bytes delivered per cycle or merely contend for the same HBM port (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). When R_{floor} is near unity, the scheduler should prefer fewer, wider memory transactions (better fusion, better layout) over more parallel blocks (Chen, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors,

2026).

Wave quantization and tail effects. Parallel splits that do not divide sequence length evenly leave tail waves with low utilization—common when KV blocks do not align with context length (Dao *et al.*, 2023; Kwon *et al.*, 2023; Davies *et al.*, 2025). Compiler scheduling should pad or bucket context lengths explicitly so tail penalties appear in CI, not only at customer ctx=8192 (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

MoE and expert parallelism. MoE layers introduce a routing axis: experts parallelize differently from attention heads (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Schedulers must avoid launching all experts when only top-k activate—parallel theory includes *conditional* parallelism gated on router output, which complicates graph capture and static NPU schedules alike (Vellaisamy *et al.*, 2026; NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Multi-hardware delta. GPU parallel scheduling uses grids/warps; CPU uses thread pools with cache affinity; NPU uses spatial arrays with fixed slot counts (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 1.4.1. Flash-Decoding reports large speedups at long context by parallelizing KV—the scheduler must still respect on-chip softmax carrier residency from Chapter 10 or parallel blocks reintroduce HBM traffic (Dao *et al.*, 2023, 2022; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; AMD, 2024a).

Parallel scheduling reviews should attach Nsight "memory throughput" and "SM busy" together—high SM busy with flat bytes/token indicates useless parallelism (Chen, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024). Decode schedulers that only maximize occupancy violate the bandwidth-aware principle this section states (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Ranking parallel axes for decode. Decode exposes four parallel axes—batch rows, query heads, KV blocks, and expert routes—and the scheduler should rank them by what each axis costs in merged state (Dao *et al.*, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024). Parallelizing over query heads is cheap until shared scores or shared KV demand a merge; parallelizing over KV blocks always ends in an online-softmax merge; parallelizing over batch rows is the only axis that adds independent work without merge traffic (Dao *et al.*, 2022; Davies *et al.*, 2025). At batch-1, batch rows vanish, which pushes decode toward head and KV parallelism and makes the merge tree a first-class scheduling cost (Vellaisamy *et al.*, 2026; Chen, 2026). The axis ranking should be recomputed per SKU because merge costs are residency costs, and residency tables differ by target (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020).

Bandwidth-saturation test for any split. Before merging a parallelization change, run the saturation test: double the split count while holding the algorithm fixed and observe whether bytes/token stays flat (Chen, 2026; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). If doubling splits raises bytes/token, the extra blocks are paying for their own reloads; if ms/token drops while bytes stay flat, the added parallelism is hiding latency productively (Davies *et al.*, 2025; Chen, 2026). The same test applies to capture groups inside graphs: a graph that launches more blocks than the memory system can feed merely serializes behind bandwidth (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Publishing the saturation curve with each kernel gives reviewers an objective reason to accept or reject a split count (Chen and YiRage Contributors, 2026; Chen, 2020).

Static versus dynamic parallelism. GPU schedulers can choose grid sizes at launch within capture buckets, while XDNA/AIE fixes the spatial mapping at compile time and CPU leaves parallelism to the runtime thread pool (NVIDIA, 2024; AMD, 2024a; Goodfellow *et al.*, 2016). Compiler IR should annotate which axes are static and which are dynamic so the same parallel plan can be legalized per backend (Chen and YiRage Contributors, 2026; Chen, 2020). A dynamic axis on GPU that becomes static on NPU changes tail behavior: the static schedule must include worst-case splits because it cannot adapt to short context (AMD, 2024b,a). Bucketing context lengths is therefore not just a graph-capture concern; it is the mechanism that keeps static schedules from wasting array capacity on tail waves (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Merge cost as a schedule term. Every split that produces partial state must pay a merge, and the schedule should treat merge bytes and merge latency as explicit costs rather than epilogue noise (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). When split count doubles, merge state roughly doubles with it, so the bandwidth-saturation test must measure the merge stage separately from the compute stage (Chen, 2026; Dao *et al.*, 2022). Compilers should annotate split boundaries with their merge tree so autotuners can compare one large split against two smaller ones on equal terms (Chen and YiRage Contributors, 2026; Chen, 2020). A merge that stays on-chip is nearly free; a merge that touches global memory can negate the split's entire benefit (SemiAnalysis, 2024; Chen, 2026).

1.5 GPU CPU NPU split

GPU CPU NPU split is how one decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP paths, and XDNA/AIE static dataflow without rewriting model Python (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). Heterogeneous fleets fail when the GPU schedule is copied to NPU—illegal DMA edges and tile overflows appear as silent CPU fallbacks or wrong results (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Backend split should share high-level passes (fusion legality, layout, tiling metadata) and fork only at emit (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Triplet regression—same model, same context buckets, GPU vs CPU vs NPU—is the acceptance test for split correctness (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). Metrics remain bytes/token and launches/token per target; FLOPs parity is irrelevant (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Version skew. GPU and NPU toolchains release on different cadences; shared IR must pin legality rules so a middle-end upgrade cannot silently break NPU emit while GPU CI stays green (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

CPU backends often serve hybrid pipelines: dynamic shapes, sampling, and small ops stay on host while GPU/NPU run fused decode cores (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). NPU backends require upfront static shape buckets; GPU backends tolerate bucketed graph capture (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Document which ops each backend owns so mode selection (Chapter 12) maps cleanly to emit paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Shared middle-end, forked back-end. The durable pattern is a shared middle-end (fusion/layout/tiling metadata) and forked backends that interpret residency classes per target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). Glow and TVM historically split early; LLM decode stacks should delay fork until after decode-specific legality checks (Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Fallback paths. When NPU legality fails, frameworks silently fall back to CPU loops—correct but catastrophically slow (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Production compilers must treat fallback as an explicit compile error unless a documented degraded mode is enabled (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026). Triplet CI should fail when GPU bytes/token improve but NPU emits empty kernels (Davies *et al.*, 2025; AMD, 2024a,b).

Heterogeneous single-node pipelines. Some deployments run attention on GPU and MLP on NPU, or prefill on GPU and decode on CPU (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023). Split theory then includes *partition* passes that cut the graph at device boundaries with explicit copy ops whose bytes count toward

goodput (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016). Hidden host-device copies dominate when partition points are chosen for convenience rather than residency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Multi-hardware delta. GPU emphasizes TMA+MMA and graphs; CPU emphasizes cache blocking and NUMA; NPU emphasizes static DMA+tiles—same IR, three residency models (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 1.5.1. A fusion plan legal on Hopper shared memory may be rejected on XDNA—split backends must surface legality errors at compile time, not as empty NPU binaries in production (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Release gates for heterogeneous builds should require artifact parity: every target emits a non-empty binary, passes numerics smoke tests, and reports bytes/token within agreed triplet tolerance (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). GPU-only green CI is insufficient for books and products that claim cross-hardware compilation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Operator ownership matrix. Backend splits are easier to maintain when the team publishes an ownership matrix: which operators emit natively on GPU, CPU, and NPU, which emit through a shared library path, and which are explicitly unsupported on each target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Empty cells in the matrix are legal only when a documented fallback exists; an undocumented empty cell is how fleet incidents begin with a silent CPU loop (AMD, 2024a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The matrix should be generated from the IR test suite so it cannot drift from what the backends actually accept (Chen, 2026, 2020). Ownership disputes—for example attention fused by the GPU backend but rejected by the NPU backend—should surface as

Table 1.1: Core theory pillars mapped to decode counters (profile per SKU).

Pillar	**Compiler question**	**Primary counter**
Tiling	Which blocks stay on-chip?	bytes/token
Fusion	Which boundaries can vanish?	bytes/token, launches/token
Layout	How are KV/weights traversed?	bytes/token
Parallel	Which axes saturate bandwidth?	ms/token, SM util.
GPU/CPU/NPU split	Where does each op emit?	triplet parity

a legality report at the shared pass level, not as a fork in model code (Chen and YiRage Contributors, 2026; AMD, 2024a).

Partition copy cost model. Heterogeneous pipelines that cut the graph at device boundaries need a cost model for the copies themselves: host-device and device-device transfers consume bandwidth and add latency, and a partition chosen for API convenience can cost more than the kernel it isolates (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). The partition pass should evaluate each candidate cut with the copy bytes charged against the same bytes/token ledger used for fusion (Chen, 2020; Chen and YiRage Contributors, 2026). When a copy is unavoidable, prefer cuts at tensors that are small relative to the compute they enable, and fuse the transfer with the following kernel where the hardware supports it (SemiAnalysis, 2024; AMD, 2024b). Release gates should show the copy bytes explicitly; hiding them behind profiler rows is how heterogeneous deployments look fused and stay slow (Davies *et al.*, 2025; Chen, 2026).

Pass ordering and golden IR diffs. The five pillars interact through ordering, so the pass pipeline itself should be versioned and tested as a unit (Chen, 2020; Chen and YiRage Contributors, 2026). Each release should capture a golden IR diff showing the sequence tiling classification, layout canonicalization, fusion contracts, parallel annotations, and backend splits (Chen, 2026; Vellaisamy *et al.*, 2026). A golden diff turns subtle ordering changes into reviewable artifacts rather than mysterious fleet regressions (Davies *et al.*, 2025; Chen, 2020). When a new pass is proposed, the review question is not whether it is clever but where it sits in the ordering and which counter it moves on batch-1 decode cells (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026).

Triplet tolerance policy. Cross-target parity needs an explicit tolerance policy: which counters must match within a percentage, which may legitimately differ because residency classes differ, and which mismatches are always failures (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). Bytes/token and launches/token should be compared per target against that target's own baseline rather than against a single global number, because an NPU that eliminates launches entirely has a different launch definition than a GPU (AMD, 2024b,a; Chen, 2026). Numerics tolerance should be defined at the attention-output level, not per op, so backend reordering that stays within output tolerance is allowed (Dao *et al.*, 2022; Davies *et al.*, 2025). Without a written policy, triplet CI becomes a source of noise or, worse, is skipped when numbers disagree (Vellaisamy *et al.*, 2026; Chen, 2026).

1.6 Key Takeaways

- **Tiling is IO theory, not FLOP theory.** FlashAttention-style blocks exist to choose which sub-blocks of weights, activations, and KV fit in registers, shared memory, L2, or NPU tile SRAM for each decode step, so the goal is cutting HBM traffic (Dao *et al.*, 2022; SemiAnalysis, 2024). Tile search must therefore be driven by decode bytes/token, not by GEMM FLOPs, or it optimizes the wrong objective (Chen, 2026; Vellaisamy *et al.*, 2026).

- **Fusion has legality, not just opportunity.** Each unfused operator boundary exports activations to HBM, but greedy fusion that ignores carrier and residency checks silently regresses decode by overflowing the on-chip budget (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Fusion theory governs *when* a boundary may legally disappear, which a compiler must verify before merging operators (Chen, 2020; Davies *et al.*, 2025).

- **Layout is a compiler transform, not a runtime afterthought.** How tensors are stored and traversed—contiguous versus blocked, paged KV tables, head-interleaved formats, TMA/vector alignment—decides whether a fusion is even reachable (Kwon *et al.*, 2023; Davies *et al.*, 2025). A wrong layout can double bytes/token without changing FLOPs, so layout assignment belongs in the pass pipeline ahead of fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

- **Parallelism must be bandwidth-aware.** Mapping work to SMs, cores, or AIE arrays—including Flash-Decoding's parallelization over KV length at batch-1—only helps when on-chip residency holds across the split (Dao *et al.*,

2023; SemiAnalysis, 2024). A parallel axis that forces activations off-chip trades latency hiding for bandwidth it cannot afford (Chen, 2026; Davies *et al.*, 2025).

- **One IR, split backends, mandatory triplet regression.** A single decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP, and XDNA/AIE static dataflow with genuinely different emit, so correctness on one target says nothing about the others (Chen and YiRage Contributors, 2026; AMD, 2024b,a). The five pillars are co-dependent—a layout change can illegalize a fusion, a fusion can force new tiles, a parallel split can break capture—so develop them as co-design profiled on decode counters, gated in CI by golden IR diffs and triplet bytes/token (Chen, 2020, 2026; Vellaisamy *et al.*, 2026).

The pillars are not independent knobs—a layout change can illegalize a fusion plan; a fusion plan can force new tile sizes; a parallel split can break graph capture (Chen and YiRage Contributors, 2026; Chen, 2020, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Treat compiler development as co-design across all five, profiled on decode counters, not as a sequence of isolated dialect tickets (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

1.7 Conclusion

Core compiler theory—tiling, fusion, layout, parallel scheduling, and heterogeneous split—is the machinery behind Chapter 12 execution modes and the MegaKernel chapters that follow (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Pass ordering slides are insufficient; profiled decode counters justify every transform (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016).

Reviewers should ask five questions before approving any compiler PR: (1) Which pillar moves? (2) Which counter must improve on BS=1 decode? (3) Which Chapter 10–11 constraint applies? (4) Does layout legalize before fusion? (5) Does triplet regression pass on GPU, CPU, and NPU? (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is "unknown," defer the patch until telemetry exists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026). These questions take minutes in review and prevent weeks of fleet debugging on batch-1 decode cells at production context buckets in CI gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Chapter 14 instantiates these principles in MLIR dialects and pass pipelines—where tiling, fusion, and layout become rewrite patterns you can test in CI (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Carry Table 1.1 forward: if a dialect change cannot name which pillar moves which counter, defer the patch (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Until then, run one decode profile per proposed pass—BS=1, ctx buckets $\{512, 2048, 8192\}$, bytes/token and launches/token on GPU and at least one non-GPU target (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). That habit converts abstract compiler theory into the measurable goodput improvements Part VI promises (Vellaisamy *et al.*, 2026; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VI continues with dialect-level mechanics; carry these pillars as the acceptance language for every rewrite pattern (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 21, 22

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 20, 21, 22

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 21, 22

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 4, 7, 8, 9, 10, 11, 12, 13, 14, 16, 17, 18, 21, 22

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 21, 22

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22